

Machine Learning

David Darras

Date de création : 05/11/2025
Dernière modification : 07/01/2025

1 Introduction

- L'**apprentissage automatique** (*machine learning*) est le processus consistant à **exploiter** un **jeu de données** (*dataset*) pour construire algorithmiquement un modèle statistique capable de généraliser ou de prédire. Des applications typiques incluent la détection de spams dans les e-mails, l'identification de transactions frauduleuses, la prévision du chiffre d'affaires, l'optimisation de processus industriels à partir de mesures de capteurs, le diagnostic de maladies par imagerie médicale ou encore la reconnaissance faciale.
- En pratique, le langage **Python** est privilégié grâce à ses nombreuses bibliothèques optimisées, notamment **NumPy** pour les calculs numériques et la manipulation de tableaux, **Pandas** pour le traitement et l'analyse de données tabulaires, **Matplotlib** pour la visualisation graphique, et **Scikit-Learn** pour l'implémentation d'algorithmes classiques de machine learning.
- Une équipe de Data Science s'articule autour de trois rôles complémentaires : le **Data Architect**, qui définit la gouvernance des données, conçoit le *Data Modeling* et garantit la scalabilité de l'infrastructure ; le **Data Engineer**, qui industrialise la collecte et la transformation des données via des pipelines ETL/ELT pour en assurer la qualité ; et enfin le **Data Scientist**, qui valorise ces données préparées en réalisant du *Feature Engineering* et en entraînant des modèles de *Machine Learning* prédictifs.
- Tout projet de Data Science suit une méthodologie rigoureuse en plusieurs étapes interdépendantes (la qualité d'une étape conditionnant la réussite de la suivante) :
 1. **Collecte des données** : Identifier et extraire les données brutes nécessaires depuis des sources hétérogènes, telles que des API (ex. : météo), des bases de données internes, des capteurs IoT ou via du scraping web.
 2. **Pré-traitement** (*Data Cleaning*) : Nettoyer les données pour les rendre exploitables. Cela implique la suppression du bruit, l'imputation des valeurs manquantes (i.e. remplacer les valeurs manquantes par la **moyenne**, la **médiane** (robuste aux outliers) ou le **mode** (valeur la plus fréquente)), la normalisation des échelles numériques ($x \leftarrow \frac{x-\mu}{\sigma}$ avec μ la moyenne et σ l'écart-type) et l'encodage des variables catégorielles.
 3. **Analyse exploratoire** (*EDA*) : Réaliser une étude statistique visuelle (histogrammes, corrélations, nuages de points) afin de comprendre la structure des données, de détecter des biais et de formuler les premières hypothèses.
 4. **Feature Engineering et Modélisation** : Créer des variables pertinentes (features) à partir des données brutes, puis sélectionner et **entraîner** l'algorithme (ex. : régression linéaire, arbre de décision) pour qu'il apprenne la fonction mathématique $y = f_{\theta}(\mathbf{x})$ liant les entrées à la cible.
 5. **Évaluation** : Tester la performance du modèle sur un jeu de données de test inédit (jamais vu durant l'entraînement) via des métriques objectives (MSE, précision, rappel) pour vérifier sa capacité à généraliser sans surajustement.
 6. **Mise en Production** (*Deployment*) : Intégrer le modèle validé dans l'environnement final (application, flux temps réel) et assurer un **monitoring** continu pour détecter la dérive des données (*data drift*) au fil du temps.

2 Apprentissage supervisé

2.1 Modélisation

- **Objectif** : Prédire une sortie y à partir d'une entrée $\mathbf{x}$ en utilisant un modèle f entraîné avec un sous-ensemble du dataset $\mathcal{D}$, appelé **jeu d'entraînement** (*training data set*) $\mathcal{D}_e$.

Exemple : On souhaite prédire la consommation électrique en kilowattheures d'un foyer à partir de trois grandeurs : la température extérieure en degrés Celsius, le nombre d'occupants et le type de logement (appartement, maison ou studio).

- **Entrée** : Vecteur de la forme $\mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_M \end{bmatrix} = [x_1 \ \dots \ x_M]^\top$ où chaque composante x_i est appelée une

caractéristique (*feature*), correspondant à une colonne (attribut) dans une base de données.

Une feature peut être une variable :

- **continue** ($x_i \in [a, b] \subset \mathbb{R}$), comme la température $x_1 \in [-90, 60]$;
- **discrète** ($x_i \in \{a_1, \dots, a_n\} \subset \mathbb{Z}$), comme le nombre d'occupants $x_2 \in \{0, \dots, 15\}$;
- **catégorielle** ($x_i \in \{a_1, \dots, a_n\}$), comme le type de logement $x_3 \in \{\text{appartement, maison, studio}\}$.

Remarque : Dans le cas catégoriel, on ne peut pas donner d'entrées brutes car les GPU travaillent avec des nombres flottants et non des chaînes de caractères. Dès lors, on peut utiliser le **Label Encoding** (ex. : *appartement* = 0, *maison* = 1, *studio* = 2), mais cela introduit une **relation d'ordre arbitraire** qui peut tromper le modèle (ici *appartement* < *maison* < *studio*) ! On préfère donc utiliser le **One-Hot Encoding** où chaque catégorie est associée à un vecteur unitaire de même longueur, bien que cela augmente la dimension (c'est-à-dire le nombre de caractéristiques) :

$$\text{appartement} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \quad \text{maison} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \quad \text{studio} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

- **Sortie** (notée y) : Il existe deux types de prédictions.
 - **Classification** : lorsque l'on cherche à prédire une catégorie, par exemple *appartement*, *maison* ou *studio*, ou encore 0 ou 1, la sortie y est appelée une **étiquette** (*label*). Une **classe** ou **catégorie** est simplement l'une des valeurs possibles que peut prendre cette étiquette.
 - **Régression** : lorsque l'on souhaite prédire une valeur réelle — comme dans notre cas avec la consommation électrique — la sortie y est appelée une **cible** (*target*).
- **Dataset** : Ensemble fini de N couples d'entrée et sortie, aussi appelés exemples étiquetés (*labeled examples*) : $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)}) \mid i = 1, \dots, N\}$. Le dataset est construit lors de la collecte de données. Par exemple, si on mesure nos trois grandeurs tous les jours avec la consommation électrique associée pendant un an, on obtient $N = 365$ couples et :

$$\mathcal{D} = \left\{ \left(\begin{bmatrix} 20.5 \\ 2 \\ \text{maison} \end{bmatrix}, 1.5 \right), \left(\begin{bmatrix} 8.0 \\ 1 \\ \text{studio} \end{bmatrix}, 0.8 \right), \dots, \left(\begin{bmatrix} 13 \\ 5 \\ \text{appartement} \end{bmatrix}, 1.2 \right) \right\}$$

Un élément du dataset $\mathcal{D}$, comme $(\mathbf{x}^{(1)}, y^{(1)}) = \left(\begin{bmatrix} 20.5 \\ 2 \\ \text{maison} \end{bmatrix}, 1.5 \right)$, est appelé une **instance**, un exemple ou encore une observation. C'est simplement une ligne de la base de données.

- **Modèle** : Fonction $f_\theta : \mathbf{x} \mapsto y$ qui prédit une valeur de sortie $\hat{y}$ à partir d'une nouvelle entrée $\mathbf{x}'$. Le modèle est obtenu en appliquant un algorithme d'apprentissage sur le jeu d'entraînement, ce qui permet d'ajuster ses paramètres θ en minimisant une **fonction de coût** $J(\theta)$ qui mesure l'erreur du modèle.

Une fois le modèle entraîné, il peut produire une prédiction. Par exemple, pour un vecteur d'entrée $\mathbf{x}' = \begin{bmatrix} 19.0 \\ 3 \\ \text{maison} \end{bmatrix}$,

le modèle pourrait renvoyer : $\hat{y} = f_\theta(\mathbf{x}') = 1.3$ kWh.

- **Hyperparamètre** : Paramètre de configuration défini **avant** l'entraînement (ex. : nombre de voisins pour k-NN, profondeur de l'arbre de décision, etc.). Il se distingue des paramètres appris automatiquement θ , car on le choisit manuellement (plutôt chiant, on évite d'en avoir beaucoup).

2.2 Évaluation et Validation du Modèle

2.2.1 Méthodologie

Pour garantir la généralisation du modèle sur de nouveaux cas :

- **Séparation des données** (*Holdout Method*) :
 - **Jeu d'entraînement** $\mathcal{D}_e$ ($\approx 75\%$) : Pour trouver $\theta^* = \operatorname{argmin}_{\theta} J(\theta)$ qui minimise la fonction de coût J .
 - **Jeu de test** $\mathcal{D}_t$ ($\approx 25\%$) : Utilisé **une seule fois** à la fin pour estimer la performance réelle.
 - **Remarque n°1** : On peut aussi utiliser un **jeu de validation** qui permet de trouver les meilleurs hyperparamètres pour notre modèle.
 - **Remarque n°2** : $\mathcal{D} = \mathcal{D}_e \cup \mathcal{D}_t$, $\mathcal{D}_e \cap \mathcal{D}_t = \emptyset$, $\operatorname{Card}(\mathcal{D}_e) = N_e$ et $\operatorname{Card}(\mathcal{D}_t) = N_t$.
- **Validation croisée** (*Cross-Validation*) : Utile pour les petits datasets. On découpe les données en K parties (*folds*) et on entraîne/teste K fois. Cela donne une moyenne de performance plus fiable.
- **Disparité des données** (*Data Mismatch*) : Quand les données d'entraînement diffèrent de celles de production. *Solution* : Utiliser un **Train-Dev Set** pour diagnostiquer si l'erreur vient du sur-apprentissage ou de la nature des données.

2.2.2 Métriques de performance

A. Pour la régression On cherche à minimiser l'erreur sous forme d'une distance :

- **MSE** (*Mean Squared Error*) : Elle pénalise fortement les grandes erreurs.

$$\text{MSE} = \frac{1}{N_t} \sum_{i=1}^{N_t} (\hat{y}^{(i)} - y_t^{(i)})^2$$

où $\hat{y}^{(i)}$ est la valeur prédite par le modèle et $y_t^{(i)}$ la valeur réelle du jeu de test (que le modèle n'a pas vue lors de l'entraînement).

- **RMSE** : Elle possède la même unité que la cible, et est donc plus interprétable. En moyenne, cela veut dire que le modèle se trompe d'environ RMSE unité.

$$\text{RMSE} = \sqrt{\text{MSE}}$$

Scénario	Comparaison des erreurs	Diagnostic
Sous-apprentissage	$\text{RMSE}_{\text{train}} \approx \text{RMSE}_{\text{test}}$ (élevées)	Modèle trop simple
Surapprentissage	$\text{RMSE}_{\text{train}} \ll \text{RMSE}_{\text{test}}$	Modèle trop complexe
Bon modèle	$\text{RMSE}_{\text{train}} \approx \text{RMSE}_{\text{test}}$ (faibles)	Bonne généralisation

- **MAE** (*Mean Absolute Error*) : Elle est plus robuste aux valeurs aberrantes (*outliers*).

$$\text{MAE} = \frac{1}{N_t} \sum_{i=1}^{N_t} |\hat{y}^{(i)} - y_t^{(i)}|$$

B. Pour la classification On utilise les quatre coefficients de la **matrice de confusion** :

- **TP** (*True Positives*) : un patient est malade et le test dit malade (succès de détection).
- **TN** (*True Negatives*) : un patient est sain et le test dit sain (succès d'exclusion).
- **FP** (*False Positives*) : un patient est sain mais le test dit malade (fausse alerte ou erreur de type I).
- **FN** (*False Negatives*) : un patient est malade mais le test dit sain (non-détection ou erreur de type II).
- **Sensibilité/Rappel** ($\text{Sensitivity/Recall} = \frac{TP}{TP+FN}$) : elle quantifie la capacité du modèle à détecter tous les cas positifs (éviter les faux négatifs).
- **Spécificité** ($\text{Specificity} = \frac{TN}{TN+FP}$) : elle quantifie la capacité du modèle à identifier correctement les cas négatifs (éviter les faux positifs).

- **Exactitude** ($\text{Accuracy} = \frac{TP+TN}{TP+FP+FN+TN}$) : proportion totale de prédictions correctes (positives et négatives) par rapport à l'ensemble des données.

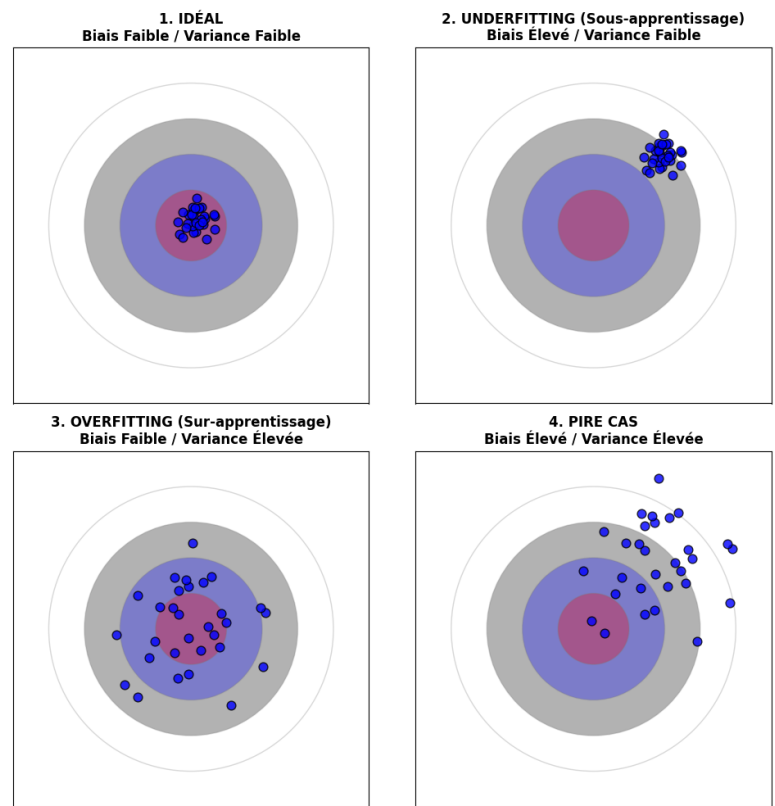
Attention : mesure trompeuse si les classes sont déséquilibrées (ex. : un modèle qui prédit toujours la classe majoritaire aura une bonne exactitude mais sera inutile).

- **Précision** ($\text{Precision} = \frac{TP}{TP+FP}$) : proportion de prédictions positives qui sont réellement correctes. Elle répond à la question : Quand le modèle prédit Oui, a-t-il souvent raison ?
- **F-score** ($\text{F1} = \frac{2 \cdot TP}{2 \cdot TP + FP + FN}$) : moyenne harmonique entre la précision et le rappel. C'est une mesure de la performance du modèle, particulièrement utile pour les classes déséquilibrées (0 pire résultat possible, 1 score parfait).

2.2.3 Diagnostic et amélioration

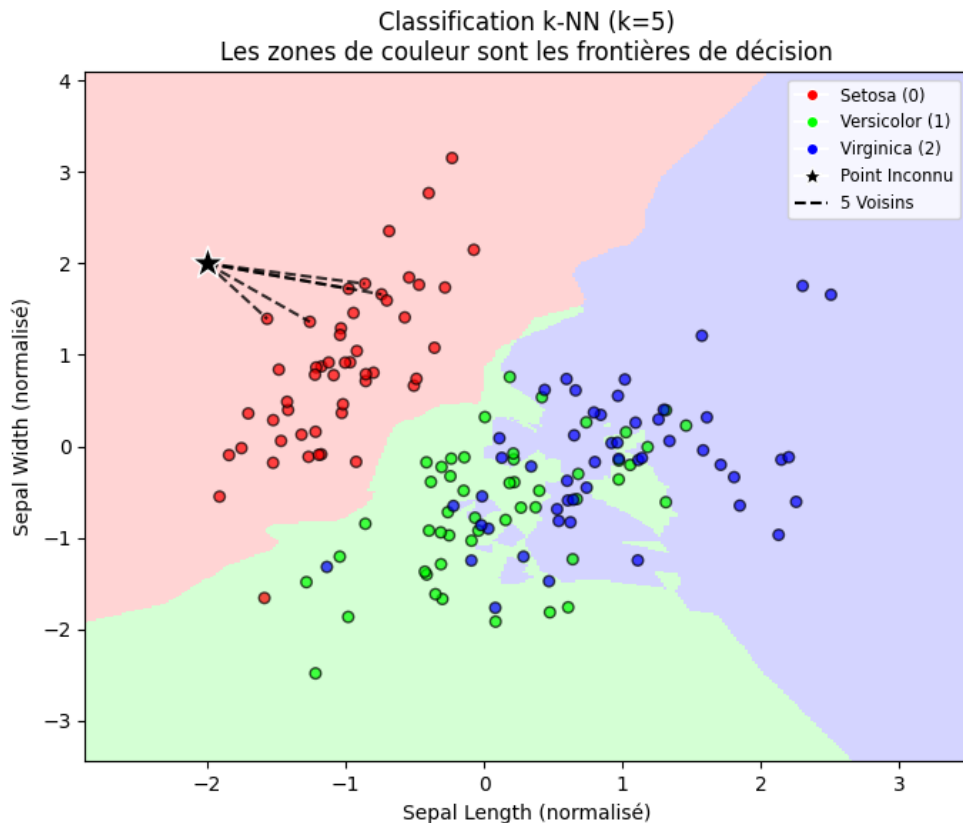
- L'erreur de généralisation d'un modèle s'exprime comme la somme de trois erreurs très différentes : le biais, la variance et l'erreur irréductible.
- **Biais** : Erreur due à des hypothèses trop simplistes dans l'algorithme d'apprentissage (ex. : essayer de modéliser une courbe complexe avec une ligne droite). Un modèle à haut biais a plus de chances de sous-ajuster les données d'entraînement.
Attention : À ne pas confondre avec le biais de la régression linéaire !
- **Variance** : Erreur due à la sensibilité excessive du modèle aux petites fluctuations (bruit) du jeu d'entraînement. Une forte variance signifie que si l'on changeait légèrement les données d'entraînement, le modèle changerait radicalement. Un modèle ayant beaucoup de degrés de liberté (comme par exemple un modèle polynomial de degré élevé) aura vraisemblablement une variance élevée, et par conséquent surajustera les données d'entraînement.
- **Erreur irréductible** : Erreur due au bruit présent dans les données. Le seul moyen de réduire cette composante de l'erreur consiste à nettoyer les données. Accroître la complexité d'un modèle va en général accroître sa variance et réduire son biais. Inversement, réduire la complexité d'un modèle accroît son biais et réduit sa variance. C'est pourquoi cet arbitrage entre biais et variance est qualifié de compromis.

- **Sous-ajustement** (*Underfitting*) : Le modèle est trop simple (**biais élevé**, faible variance). Il ne capture pas la structure des données.
Solutions : Modèle plus complexe, ajout de *features*, réduction de la régularisation.
- **Sur-ajustement** (*Overfitting*) : Le modèle est trop complexe (**variance élevée**, faible biais). Il apprend le bruit par cœur.
Solutions : Simplifier le modèle, plus de données, ou utiliser la **régularisation**.
- **Régularisation** : Technique consistant à contraindre le modèle (réduction des degrés de liberté) pour éviter le sur-apprentissage, souvent en ajoutant une pénalité sur les poids (Lasso L_1 , Ridge L_2).



2.3 k-NN — Les k plus proches voisins (*k Nearest Neighbors*)

- **Hypothèse** : Les données ayant des caractéristiques proches partagent la même étiquette.
- **Objectif** : On souhaite entraîner notre modèle pour qu'il puisse déterminer, à partir de la longueur et de la largeur d'un sépale (les features en entrée), si notre iris est une setosa, versicolor ou virginica (les catégories possibles en sortie). Dans notre cas, il s'agit de classification mais cela fonctionne aussi pour la régression.
- **Remarque** : Le modèle mémorise tout le jeu d'apprentissage, il n'y a **pas de phase d'entraînement** et donc pas d'optimisation pour trouver θ^* .
- **Hyperparamètres** :
 - **Le nombre de voisins k** qui contrôle le compromis biais/variance. Si trop petit, on a un faible biais, une forte variance (overfitting). Si trop grand, on a un biais élevé, une faible variance (underfitting). Pour choisir k , on utilisera la **méthode du coude** (*elbow method*) : on teste une plage de valeurs pour k (1, 2, etc.) en calculant l'erreur associée (la MSE), puis on sélectionne le point où la courbe forme un coude (la baisse d'erreur ralentit brutalement), ce qui correspond au meilleur compromis. En général, on prend $k = 3$ ou 5.
 - **La métrique de distance** : On prend généralement la norme **euclidienne** (L_2) : $\|\mathbf{u} - \mathbf{v}\|_2 = \sqrt{\sum_{i=1}^M (u_i - v_i)^2}$, adaptée aux données physiques. Ou bien celle de **Manhattan** (L_1) : $\|\mathbf{u} - \mathbf{v}\|_1 = \sum_{i=1}^M |u_i - v_i|$, adaptée aux déplacements sur une grille ou aux hautes dimensions.
- **Attention** : Il est impératif de **normaliser**, sinon les caractéristiques avec de grandes échelles dominent le calcul de distance. De plus, on a une complexité algorithmique de prédiction en $O(N \times M)$ (où N est le nombre d'échantillons et M le nombre de caractéristiques). L'algorithme est très simple mais pas très *scalable* (lent si le jeu de données est volumineux), même si l'on utilise l'astuce du KD-Tree pour passer en $O(\log(N) \times M)$.
- **Principe** : On a une entrée normalisée $\mathbf{x}' = [-2 \ 2]^\top$. On calcule sa distance par rapport à tous les points du dataset et on sélectionne les k plus proches (distances moindres).
 - **Classification** : Vote majoritaire. Soit **uniforme** : $y = \operatorname{argmax}_c \sum_{i=1}^k \mathbb{I}(y^{(i)} = c)$. Soit **pondéré** : où on ajoute un poids : $w^{(i)} = \frac{1}{\|\mathbf{x}' - \mathbf{x}^{(i)}\|}$, utile si les classes sont déséquilibrées ou si un voisin est très proche.
 - **Régression** : TODO



2.4 Régression linéaire

- **Objectif** : Prédire une valeur réelle $\hat{y}$ à partir d'une nouvelle entrée $\mathbf{x}$ et à l'aide de la fonction f_{θ} , où θ est un vecteur de paramètres à déterminer :

$$\hat{y} = f_{\theta}(\mathbf{x}) = \theta^{\top} \mathbf{x} = \theta_0 + \theta_1 x_1 + \dots + \theta_n x_n$$

Le terme constant θ_0 est appelé le biais (*intercept*) et les autres sont les coefficients de pondération. Pour simplifier la notation vectorielle, on pose toujours $x_0 = 1$.

- **Fonction de coût** : Basée sur la MSE.

$$J(\theta) = \frac{1}{2N_e} \sum_{i=1}^{N_e} (\theta^{\top} \mathbf{x}^{(i)} - y^{(i)})^2.$$

Pour trouver le minimum, il suffit de **dériver** la fonction (calculer le gradient) et de trouver le point où elle s'annule. La fonction de coût étant **convexe**, il existe un **unique minimum global** ; il est donc impossible de rester piégé dans un minimum local, comme on le verra plus tard pour d'autres algorithmes.

- **Solution analytique** : Elle est donnée par l'équation normale :

$$\theta^* = (X^{\top} X)^{-1} X^{\top} y$$

où X est la matrice des features (de taille $N \times (M + 1)$) et y le vecteur cible.

Remarque n°1 : Le terme $(X^{\top} X)^{-1} X^{\top}$ porte le doux nom de **pseudo-inverse de Moore-Penrose**.

Remarque n°2 : Une ligne de X est un échantillon, une colonne est une feature. Il ne faut pas oublier d'ajouter la colonne de 1 au début pour le biais !

- **Exemple** : Prédire la consommation électrique.

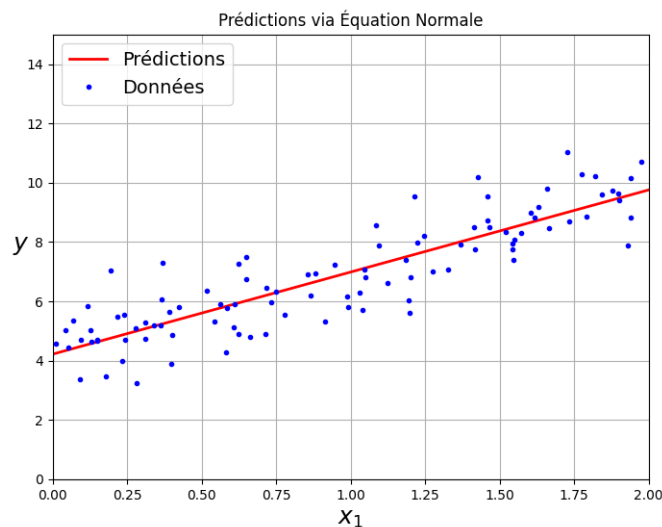
$$X = \begin{bmatrix} 1 & 20.5 & 2 & 0 & 0 \\ 1 & 0.8 & 1 & 1 & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & 13 & 5 & 0 & 1 \end{bmatrix} \quad \text{et} \quad y = \begin{bmatrix} 1.5 \\ 0.8 \\ \vdots \\ 1.2 \end{bmatrix}$$

Les colonnes de X sont : [biais, température, occupants, estAppart ?, estMaison ?].

Attention : Pour l'encodage catégoriel (*One-Hot*), si on a 3 catégories (appartement, maison, studio), on ne met que 2 colonnes binaires (ici, on a retiré studio). Si on mettait les 3, la somme des colonnes vaudrait 1 (colinéarité avec le biais), ce qui rendrait la matrice $X^{\top} X$ non inversible.

- **Remarque** : Théoriquement, l'équation normale trouve la solution sans normaliser les données (ex. : on laisse la température à 20.5). Cependant, il est recommandé de le faire pour éviter des problèmes de stabilité numérique. Si les features ont des ordres de grandeur trop différents, la matrice peut devenir mal conditionnée et les erreurs d'arrondi machine faussent l'inversion. De plus, la normalisation est obligatoire si l'on passe à une résolution par *descente de gradient* !

- **Illustration avec une seule feature et un biais** :

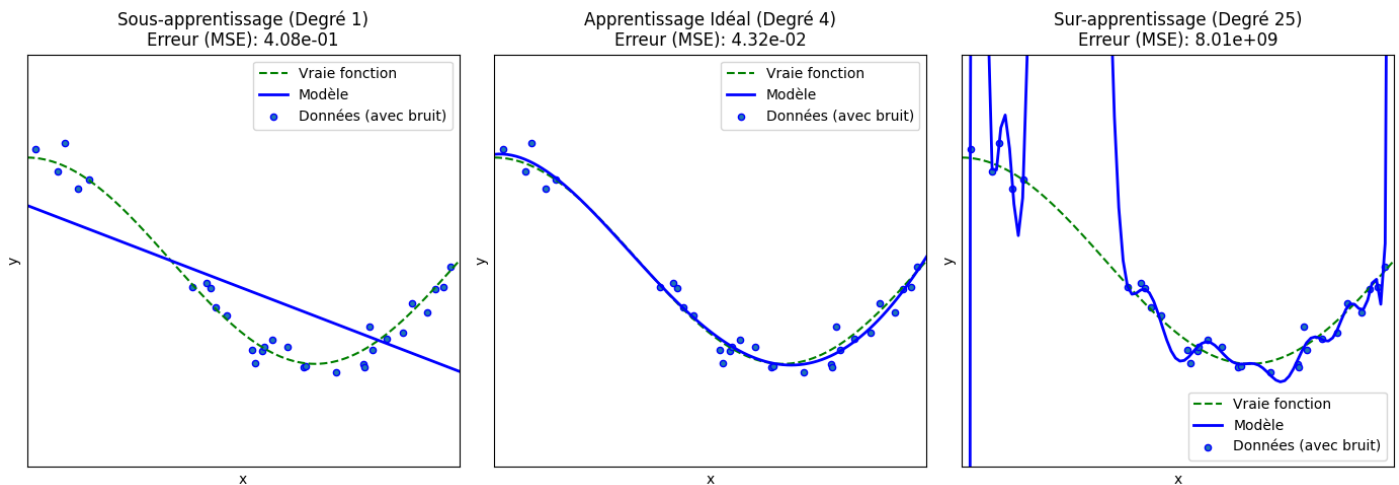


- **Limites de l'approche analytique :**

- **Complexité algorithmique :** L'inversion de la matrice $X^\top X$ est coûteuse ($O(M^{2.4})$ à $O(M^3)$). Doubler le nombre de caractéristiques M multiplie le temps de calcul par 5 à 8.
- **Contrainte mémoire :** Le dataset complet doit résider en **RAM** pour effectuer les calculs matriciels, ce qui limite le passage à l'échelle (*scalability*).
- **Instabilité numérique :** Si les *features* sont fortement corrélées (colinéarité) ou mal normalisées, la matrice devient **mal conditionnée**. Cela amplifie les erreurs d'arrondi et peut rendre la **matrice singulière** (non inversible).
- **Manque de flexibilité :** La solution analytique est exclusive à la régression linéaire simple. Dès que l'on ajoute une régularisation complexe ou que l'on utilise une fonction de coût non quadratique, elle n'est plus applicable.

À l'inverse, la **prédiction** reste extrêmement rapide car elle ne nécessite qu'un simple produit scalaire.

- **Double Descente :** Contrairement à la courbe classique en "U", en régime fortement *sur-paramétré* ($M \gg N$), l'erreur de test peut diminuer à nouveau après avoir augmenté. Passé le **seuil d'interpolation** ($M \approx N$), le modèle dispose de tant de degrés de liberté qu'il converge vers la solution de **norme minimale** (la plus "lisse"). Cette sélection agit comme une régularisation implicite qui améliore la généralisation.
- **Régression polynomiale :** Tant qu'on reste **linéaire par rapport aux paramètres** θ , on peut mettre n'importe quoi pour le vecteur d'entrée $\mathbf{x}$, comme $\mathbf{x} = \begin{bmatrix} 1 \\ x \\ x^2 \end{bmatrix}$. Attention toutefois : si on utilise un polynôme de haut degré, il est impératif d'utiliser la **régularisation** (Ridge, Lasso) pour contraindre les poids (c'est-à-dire les réduire ou les mettre à zéro) afin d'éviter le sur-apprentissage !



2.5 Descente de gradient

- L'objectif reste le même que pour la régression linéaire. Or, ici, on ne va pas calculer directement θ^* d'un seul coup : on va utiliser un **algorithme d'optimisation itératif**.
- **Algorithme :**

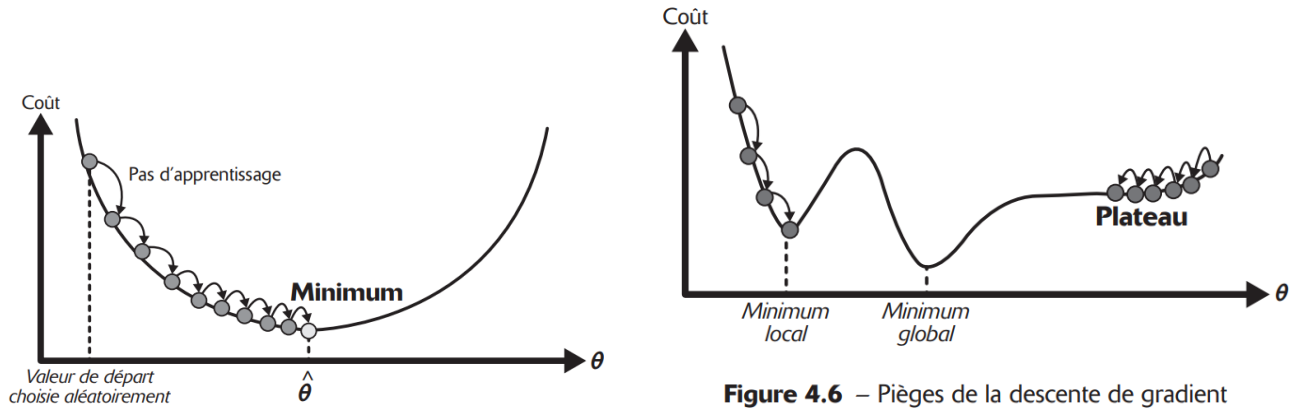
1. **Initialisation :** On commence avec des valeurs aléatoires pour $\theta^{(0)}$ (position au hasard sur la courbe de la fonction de coût J).
2. **Calcul du gradient $\nabla_{\theta} J$:** On calcule la pente de la fonction de coût par rapport à chaque paramètre θ_j avec une dérivée partielle. Le vecteur gradient pointe vers la montée la plus raide. Dans le cas de la MSE, on a :

$$\nabla_{\theta} J(\theta^{(k)}) = \frac{2}{N_e} X^\top (X\theta^{(k)} - y)$$

3. **Mise à jour :** On fait un pas dans la direction opposée au gradient pour descendre :

$$\theta^{(k+1)} = \theta^{(k)} - \eta \nabla_{\theta} J(\theta^{(k)})$$

4. On répète jusqu'à ce que le gradient soit quasi nul (convergence vers un minimum).



Remarque n°1 : Cette version Batch utilise tout le dataset X pour calculer le gradient à chaque étape. Elle est précise mais lente sur les très gros datasets.

Remarque n°2 : Pour des fonctions de coût J complexes, l'algorithme peut se piéger dans un **minimum local** (un petit creux) ou stagner sur un plateau.

- **Taux d'apprentissage η (Learning Rate) :** C'est un hyperparamètre crucial qui détermine la taille des pas.
 - **Trop petit :** L'algorithme converge, mais prend un temps infini (beaucoup d'itérations).
 - **Trop grand :** L'algorithme risque de sauter par-dessus la vallée, d'osciller, voire de diverger (remonter la pente).

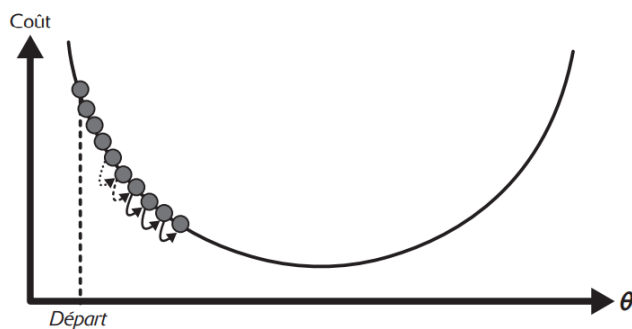


Figure 4.4 – Le taux d'apprentissage est trop petit

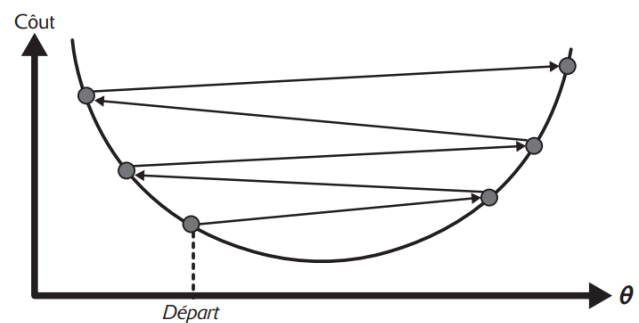


Figure 4.5 – Le taux d'apprentissage est trop élevé

- **Importance de la normalisation :** Si les features ont des échelles très différentes (ex: $x_1 \in [0, 1]$ et $x_2 \in [0, 1000]$), la fonction de coût aura la forme d'un bol très allongé. La descente de gradient va osciller et peiner à trouver le fond. **Il est impératif d'utiliser le *StandardScaler* avant une descente de gradient** pour rendre le bol bien rond et accélérer la convergence.
- **Variantes (Stochastique et Mini-Batch) :**
 - **Batch :** Utilise tout le dataset à chaque étape (lent, trajectoire stable).
 - **Stochastique (SGD) :** Utilise une seule instance choisie au hasard à chaque étape (très rapide, trajectoire très agitée, échappe aux minima locaux).
 - **Mini-Batch :** Compromis (utilise un petit paquet d'instances). C'est la méthode la plus utilisée en Deep Learning.
 - **Adam :** Optimiseur adaptatif. Ajuste le learning rate pour chaque paramètre et utilise l'inertie (momentum). C'est souvent le choix par défaut car il converge plus vite.

2.6 Régression logistique

- Dans le cas où le modèle produit un réel et qu'on souhaite déterminer une catégorie binaire $\{0, 1\}$ (ex. : $\{\text{sain}, \text{malade}\}$, $\{\text{mail}, \text{spam}\}$), on utilisera la **sigmoïde** :

$$\sigma(z) = \frac{1}{1 + e^{-z}} \in [0, 1]$$

Elle permet de ramener la valeur entre 0 et 1. On applique ensuite un seuil de décision : si le résultat est > 0.5 , c'est un spam ; si < 0.5 , c'est un mail.

- Dans le cas avec K classes, on obtient un score brut pour chaque catégorie (ex. : 2 pour *appartement*, 4.5 pour *maison*, -1 pour *studio*). On transforme ces scores en probabilités avec la fonction **softmax** :

$$\sigma \left(\begin{bmatrix} z_1 \\ \vdots \\ z_K \end{bmatrix} \right)_c = \frac{e^{z_c}}{\sum_{k=1}^K e^{z_k}} \in [0, 1], \quad \sum_{k=1}^K \sigma(\mathbf{z})_k = 1$$

Exemple de résultat : 0.08 (8%) pour *appartement*, 0.91 (91%) pour *maison* et 0.01 (1%) pour *studio*. Les valeurs sont comprises entre 0 et 1 et leur somme vaut 1. La prédiction finale est donc *maison* (la classe ayant la probabilité la plus élevée).

- **Fonction de coût et optimisation** : Contrairement à la régression linéaire :

– On n'utilise plus la MSE mais la **log-loss** (cas binaire) :

$$J(\theta) = -\frac{1}{N_e} \sum_{i=1}^{N_e} \left[y^{(i)} \log(\sigma(\hat{y}^{(i)})) + (1 - y^{(i)}) \log(1 - \sigma(\hat{y}^{(i)})) \right]$$

– Pour le cas multi-classe, on utilise la **cross-entropy** :

$$J(\Theta) = -\frac{1}{N_e} \sum_{i=1}^{N_e} \sum_{k=1}^K y_k^{(i)} \log(\sigma(\hat{y}^{(i)})_k)$$

– La fonction de coût n'admet pas de solution analytique simple : on optimise θ par descente de gradient (ou variantes stochastiques / mini-batch).

2.7 Arbre de décision

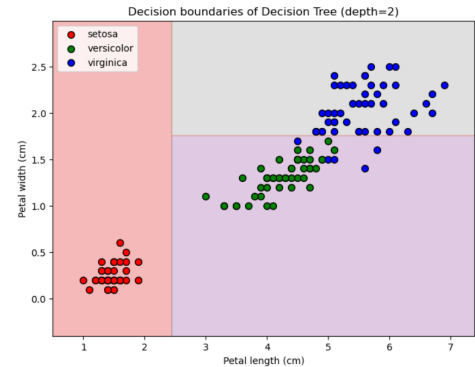
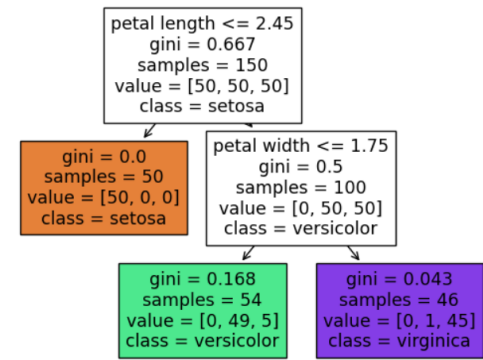
- **Objectif** : Diviser les données en groupes de plus en plus homogènes (purs) sans utiliser de paramètre θ .
- **Principe** : Dans chaque nœud interne, une caractéristique est évaluée et comparée à un seuil. Si la valeur est inférieure au seuil, on suit la branche gauche ; sinon, la branche droite. Lorsqu'on atteint une feuille, le modèle fournit une prédiction par rapport à la classe majoritaire (classification) ou une valeur réelle (régression).
- **Exemple** : Classifier des iris avec la longueur et largeur du sépale.

- L'indice de **Gini** mesure l'impureté d'un nœud. Un nœud est pur ($G_i = 0$) si toutes les instances appartiennent à la même classe. Si $G_i = 0.5$, le mélange est équilibré entre deux classes :

$$G_i = 1 - \sum_{k=1}^K p_{i,k}^2 \in [0, 1]$$

où $p_{i,k}$ est la proportion d'instances de classe k dans le $i^{\text{ème}}$ nœud et K le nombre total de classes. On peut aussi utiliser l'entropie : $H_i = -\sum_{k=1}^K p_{i,k} \log_2(p_{i,k})$

- **samples = 54** indique le nombre total d'instances dans le nœud.
- **value = [0, 49, 5]** indique le nombre d'instances de chaque catégorie : 0 Setosa, 49 Versicolor et 5 Virginica.
- **class = Versicolor** indique la catégorie majoritaire dans le nœud. Par exemple, si la longueur du pétale ≤ 2.45 cm et la largeur ≤ 1.75 cm, la fleur est classée comme Versicolor.
- **Remarque** : On limite la profondeur de l'arbre (ici à 2) pour éviter le sur-apprentissage (c'est un hyperparamètre). Si l'arbre apprenait tout le dataset par cœur, on obtiendrait des feuilles pures (une seule instance), ce qui empêcherait la généralisation.



- **Algorithme de CART** (*Classification And Regression Tree*) : L'algorithme fonctionne de manière **gloutonne** (*greedy*) : au nœud racine, il cherche la meilleure paire (caractéristique k , seuil t_k) qui divise le jeu de données en deux sous-ensembles les plus purs possibles (pondérés par leur taille). Il répète ce processus récursivement pour chaque sous-ensemble.
- La **fonction de coût** J que l'algorithme tente de **minimiser** à chaque division est :

$$J = \frac{m_{\text{gauche}}}{m} G_{\text{gauche}} + \frac{m_{\text{droite}}}{m} G_{\text{droite}}$$

où :

- $G_{\text{gauche/droite}}$ mesure l'impureté (Gini) du sous-ensemble gauche/droite.
- $m_{\text{gauche/droite}}$ est le nombre d'échantillons dans le sous-ensemble gauche/droite.
- m est le nombre total d'échantillons dans le nœud parent.

L'algorithme s'arrête lorsqu'il atteint la profondeur maximale (**max_depth**), lorsqu'il ne peut plus réduire l'impureté, ou si le nœud contient moins de **min_samples_split** échantillons.

- **Forêts aléatoires** (*Random Forest*) :
 - Méthode d'ensemble (*Bagging*).
 - **Principe** : On entraîne plusieurs arbres en parallèle sur des sous-parties aléatoires du dataset.
 - **Avantage** : Réduit la variance (overfitting) par rapport à un arbre unique, sans augmenter le biais.

3 Apprentissage non-supervisé

Dans l'apprentissage supervisé, on fournit au modèle des entrées $\mathbf{x}$ avec les sorties y associées pour qu'il apprenne. L'objectif est de prédire une valeur $\hat{y}$ pour de nouvelles entrées $\mathbf{x}'$.

Dans l'apprentissage non supervisé, on fournit au modèle uniquement les entrées. Le modèle doit alors identifier lui-même des structures ou regrouper les données selon leurs similarités.

3.1 Algorithme des k-Moyennes (*k-Means*)

- **Objectif** : Partitionner le dataset en k groupes distincts (*clusters*) de telle sorte que les données d'un même groupe soient similaires (faible distance intra-cluster) et que les groupes soient distincts les uns des autres. Contrairement au k-NN (supervisé), le K-Means est un algorithme d'**apprentissage non supervisé**. On ne dispose pas des étiquettes y qui donnent la classe !

- **Algorithme itératif de Lloyd-Forgy** :

1. **Initialisation** : On place aléatoirement k points appelés **centroïdes** $(\mu^{(1)}, \dots, \mu^{(k)})$.

Remarque : On utilise souvent la méthode k-means++ qui place les centroïdes initiaux loin les uns des autres pour accélérer la convergence.

2. **Affectation** : On attribue chaque point de données $x^{(i)}$ au cluster du centroïde le plus proche (distance euclidienne).

$$c^{(i)} := \operatorname{argmin}_{l \in \{1, \dots, k\}} \sum_{j=1}^M (x_j^{(i)} - \mu_j^{(l)})^2$$

Exemple : si $x^{(3)}$ est plus proche de $\mu^{(2)}$, alors $c^{(3)} = 2$ et donc $x^{(3)} \in C_2$.

3. **Mise à jour** : On déplace chaque centroïde vers le **barycentre** (la moyenne) des points qui lui ont été affectés.

$$\mu^{(l)} := \frac{1}{|C_l|} \sum_{x \in C_l} x$$

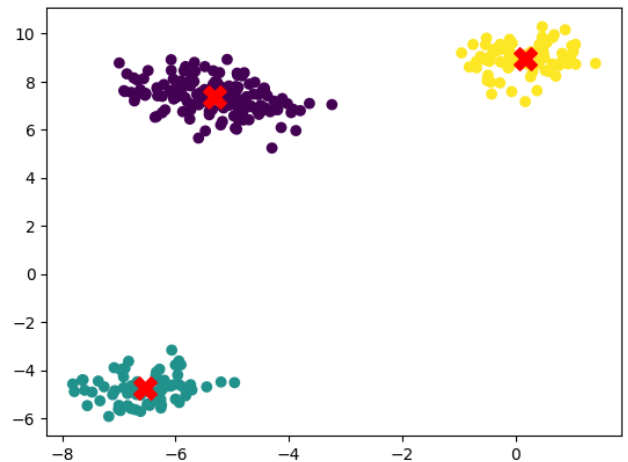
Ou élément par élément pour la dimension j : $\mu_j^{(l)} := \frac{1}{|C_l|} \sum_{x \in C_l} x_j$

4. **Répétition** : On répète les étapes 2 et 3 jusqu'à ce que les centroïdes ne bougent plus (convergence).

- **Fonction de coût** : On cherche à minimiser l'**inertie** (*Within-Cluster Sum of Squares*), c'est-à-dire la somme des distances au carré entre chaque point et son centroïde :

$$J = \sum_{i=1}^N \sum_{j=1}^M (x_j^{(i)} - \mu_j^{(c^{(i)})})^2$$

L'algorithme converge toujours, mais peut tomber dans un *minimum local* (solution sous-optimale) selon l'initialisation. C'est pourquoi on lance l'algorithme plusieurs fois pour garder le meilleur résultat.



- **Choix du nombre de clusters k** : (Encore un hyperparamètre)

- **Méthode du coude** (*Elbow Method*) : On trace l'inertie en fonction de k . L'inertie diminue toujours quand k augmente (si $k = N$, inertie = 0). On cherche le point d'inflexion (le coude) où le gain d'inertie commence à devenir négligeable.
- **Score de Silhouette** : Plus robuste. Il mesure à quel point un objet est similaire à son propre cluster comparé aux autres clusters.

$$s = \frac{b - a}{\max(a, b)}$$

Où a est la distance intra-cluster moyenne et b la distance moyenne au cluster voisin le plus proche. $s \in [-1, 1]$. Plus on est proche de 1, mieux c'est.

- **Limitations** :

- Nécessité de choisir le nombre de clusters k manuellement.
- Sensible aux **outliers** (valeurs aberrantes), qui peuvent déplacer significativement le centroïde (la moyenne).

- Suppose que les clusters sont **convexes** (sphériques), de **variance équivalente** (taille similaire) et **isotropiques** (s'étalent uniformément dans toutes les directions).
- Échoue sur des formes géométriques complexes (ex. : bananes, cercles concentriques) ou des densités variées, car l'algorithme cherche à minimiser la variance intra-cluster en se basant uniquement sur la distance euclidienne.
- *Solutions alternatives* : utiliser **DBSCAN** (basé sur la densité) pour les formes arbitraires, ou les **GMM** (Gaussian Mixture Models) qui généralisent K-Means en acceptant des formes ellipsoïdales et des probabilités d'appartenance.

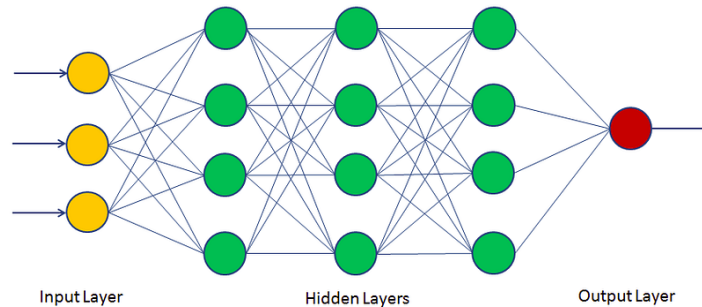
- **Variante (Mini-Batch K-Means)** : Pour les très gros datasets, on utilise une version qui ne met à jour les centroïdes qu'avec un petit échantillon aléatoire de données à chaque étape. C'est beaucoup plus rapide mais très légèrement moins précis.

3.2 Analyse en Composantes Principales (PCA)

- **Objectif** : réduire la dimensionnalité des données (nombre de features) tout en préservant le maximum d'information (variance).
- **Principe** : projeter les données sur de nouveaux axes orthogonaux (les composantes principales).
- **Utilité** :
 - **Visualisation** : projeter des données complexes (ex. : 100 dimensions) en 2D ou 3D pour voir des groupes.
 - **Pré-traitement** : enlever le bruit et réduire le temps de calcul avant d'appliquer un algorithme comme K-Means.

4 Introduction au *Deep Learning*

Un **réseau de neurones** est un assemblage de fonctions simples (des couches) que l'on compose pour créer une fonction complexe capable d'apprendre.



4.1 Les briques de base

- **Couche Linéaire (Fully Connected)** : Pour une entrée $\mathbf{x}$ (vecteur de taille n), la sortie $\mathbf{z}$ (vecteur de taille m) est :

$$\mathbf{z} = W\mathbf{x} + \mathbf{b}$$

Où W est la **matrice des poids** (taille $m \times n$) initialisée avec de petites valeurs aléatoires et $\mathbf{b}$ le **biais** (taille m). Ce sont ces paramètres $\theta = \{W, b\}$ que le réseau apprend.

- **Fonctions d'activation** : Si l'on n'enchaîne que des couches linéaires, le résultat final reste une fonction linéaire (inutile pour des problèmes complexes). On introduit de la **non-linéarité** après chaque couche linéaire grâce à une fonction d'activation σ appliquée élément par élément :
 - **ReLU (Rectified Linear Unit) / Fonction de Heaviside** : $\sigma(z) = \max(0, z)$. C'est la plus utilisée aujourd'hui (depuis 2012). Elle est simple à calculer, évite la disparition du gradient pour les valeurs positives, mais sa dérivée est nulle pour $z < 0$.
 - **Sigmoïde** : $\sigma(z) = \frac{1}{1+e^{-z}}$. Elle écrase les valeurs entre $[0, 1]$. Historiquement utilisée, elle est aujourd'hui réservée à la couche de sortie pour la classification binaire (probabilité).

- **Softmax** : $\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$. Utilisée en **sortie** pour la classification multi-classes. Elle transforme un vecteur de scores bruts (logits) en une distribution de probabilité (somme = 1).

La transmission de cette entrée est connue sous le nom de **propagation en avant** (*forward*).

- **Apprentissage par rétropropagation** (*Back-propagation*) : Ce processus est l'algorithme clé pour entraîner les réseaux de neurones en ajustant leurs paramètres θ . Il suit un cycle itératif composé de quatre étapes :
 - **Propagation vers l'avant** (*Forward pass*) : On calcule la prédiction $\hat{y} = f_{\theta}(x)$ en traversant le réseau de l'entrée vers la sortie [cite: 1151].
 - **Calcul du coût** : On mesure l'erreur J entre la prédiction et l'étiquette réelle. Si la sortie correspond à l'étiquette, aucun ajustement n'est nécessaire ; sinon, il faut corriger les poids.
 - **Propagation vers l'arrière** (*Backward pass*) : On calcule le gradient de l'erreur par rapport à chaque paramètre $\nabla_{\theta} \mathcal{L}$.
 - **Mise à jour (Update)** : On ajuste les poids pour minimiser l'erreur en faisant un pas dans la direction opposée au gradient.
- **Règle de la chaîne** (*Chain Rule*) : Pour calculer l'influence de chaque paramètre θ_i sur l'erreur finale, on utilise la règle de la chaîne qui permet de décomposer la dérivée de manière récursive :

$$\frac{\partial J}{\partial \theta_i} = \frac{\partial J}{\partial z_i} \times \frac{\partial z_i}{\partial \theta_i}$$

Le signal d'erreur remonte ainsi couche par couche, de la sortie vers l'entrée.

- **Limites de l'optimisation** : Contrairement à la régression linéaire qui possède un unique minimum global, l'optimisation d'un réseau de neurones est plus complexe. L'algorithme peut rester piégé dans un **minimum local** ou stagner sur un **plateau**, ne garantissant pas ainsi une convergence vers l'optimum global.

4.2 MLP – Perceptron Multicouche (*MultiLayer Perceptron*)

- **Architecture** : Empilement séquentiel de couches linéaires et d'activations. On note L la profondeur du réseau (nombre de couches).

$$\hat{y} = f_{\theta}(\mathbf{x}) = f_L(\dots \sigma_2(f_2(\sigma_1(f_1(\mathbf{x})))) \dots)$$

- **Hyperparamètres** :
 - **Profondeur** : Nombre de couches cachées.
 - **Largeur** : Nombre de neurones dans chaque couche (souvent des puissances de 2 : 128, 256, 1024...). Ces dimensions intermédiaires ne sont pas interprétables humainement.
- **Théorème d'Approximation Universelle** : Théoriquement, un MLP avec une seule couche cachée suffisamment large (nombre de neurones infini) peut approximer n'importe quelle fonction continue avec une précision arbitraire. *Note* : Ce théorème est rassurant mathématiquement mais ne dit pas comment trouver les paramètres (l'entraînement reste difficile).
- **Limitations pour l'image** : Pour traiter une image (ex: MNIST, chiffres manuscrits 28×28 pixels) avec un MLP, il faut "aplatir" (*flatten*) l'image en un vecteur colonne de taille 784.
Problème : On perd toute la structure spatiale (un pixel en haut à gauche se retrouve très loin d'un pixel en bas à droite dans le vecteur). Le MLP ne "voit" pas les formes, il voit juste une suite de nombres.

4.3 CNN – Réseaux de Neurones Convolutifs (*Convolutional Neural Network*)

- **Intuition** : Au lieu de connecter chaque pixel à chaque neurone (trop de paramètres, perte de structure), on utilise des filtres locaux qui parcourent l'image pour détecter des motifs (bords, textures, formes).
- **Convolution** : L'opération mathématique centrale. On fait glisser une petite matrice appelée noyau (*kernel*) sur l'image. Cela préserve la cohérence spatiale.

Exemple : Soit une image I de taille 4×4 : $I = \begin{bmatrix} 1 & 2 & 0 & 1 \\ 3 & 1 & 2 & 2 \\ 0 & 1 & 3 & 1 \\ 2 & 0 & 1 & 0 \end{bmatrix}$ et un kernel (filtre) K de taille 2×2 : $K = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$.

La convolution consiste à :

1. Superposer le kernel sur une partie de l'image.
2. Multiplier chaque élément du kernel par l'élément correspondant de l'image.
3. Additionner tous ces produits pour obtenir un seul nombre.
4. Déplacer le kernel (stride = 1) et répéter jusqu'à couvrir toute l'image.

La taille de la sortie O est : $O = \frac{N-F}{S} + 1 = \frac{4-2}{1} + 1 = 3$, avec N la taille de l'image et F la taille du kernel, donc O est de taille 3×3 .

Après avoir appliqué la convolution à toutes les positions, on obtient : $O = I * K = \begin{bmatrix} 0 & 0 & -2 \\ 3 & 0 & 1 \\ 2 & 3 & 2 \end{bmatrix}$.

- **Architecture typique :**

1. **Couches convolutives :** Extraction de caractéristiques (*feature maps*).
 2. **Pooling (Mise en commun) :** Réduction de la dimension (ex: Max Pooling prend le maximum dans un carré 2×2) pour rendre le modèle invariant aux petites translations et réduire le coût de calcul.
 3. **Flatten + MLP :** À la fin, on aplatit les features extraites pour les passer dans un classifieur classique (MLP) qui donne la prédiction finale.
- **Zero-padding :** Hyperparamètre qui permet de contrôler la dimension spatiale des volumes de sortie. On ajoute une bordure de zéros autour de l'image pour conserver sa taille originale ou ajuster le champ réceptif.

5 Références

